

CSE 1113

Computer Programming

Sumaya Kazary

Assistant Professor

Department of Computer Science and Engineering
Dhaka University of Engineering & Technology, Gazipur

Acknowledgement

Thanks to the authors of all the books and online tutorials used in this slide.

User Defined Function Part - 2

Types of User-defined Functions in C Programming

- ❑ For better understanding of arguments and return type in functions, user-defined functions can be categorised as:
 - Function with no arguments and no return value
 - Function with no arguments and return value
 - Function with arguments but no return value
 - Function with arguments and return value
- ❑ Let's take an example *to find whether a number is prime or not* using above 4 categories of user defined functions.

Function with no arguments and no return value

```
void prime();  
int main(){  
    prime();  
    return 0;
```

There is no return value to calling function main(). Hence, return type of prime() is void

```
void prime() {  
    int num,i,flag=0;  
    printf("Enter positive integer enter to check:\n");  
    scanf("%d",&num);  
    for(i=2;i<=num/2;++i){  
        if(num%i==0){  
            flag=1;  
            break;}  
    }  
    if (flag==1) printf("%d is not prime",num);  
    else printf("%d is prime",num);  
}
```

Function with no arguments but return value

```
int prime();  
int main(){  
    int p;  
    p=prime();  
    if(p) printf("\n Inputted Value is not prime");  
    else  
        printf("\n Inputted Value is prime");
```

```
int prime() {  
    int num, i, flag=0;  
    printf("Enter positive integer to check:\n");  
    scanf("%d",&num);  
    for(i=2;i<=num/2;++i){  
        if(num%i==0){  
            flag=1; }  
    }  
    return flag;  
}
```

There is no argument passed to prime() function But, the value of flag returned to main() function.

Function with arguments and no return value

```
void prime(int n);  
int main(){  
    int num;  
    printf("Enter positive enter to check:\n");  
    scanf("%d",&num);  
    prime(num);  
    return 0;  
}
```

```
void prime(int n){  
    int i, flag = 0;  
    for(i=2; i<=n/2; ++i){  
        if(n%i==0){ flag = 1;  
                    break; }  
    }  
    if(flag == 1)  
        printf("%d is not prime",n);  
    else  
        printf("%d is prime", n);  
}
```

Here, argument is passed to user-defined function but, value is not returned from it to calling function.

Function with arguments and a return value

```
int prime(int n);  
int main() {  
    int num,p=0;  
    printf("Enter positive enter to check:\n");  
    scanf("%d",&num);  
    p=prime(num);  
    if(p==1) printf("%d is not prime",num);  
    else  
        printf("%d is prime",num);  
    return 0;  
}
```

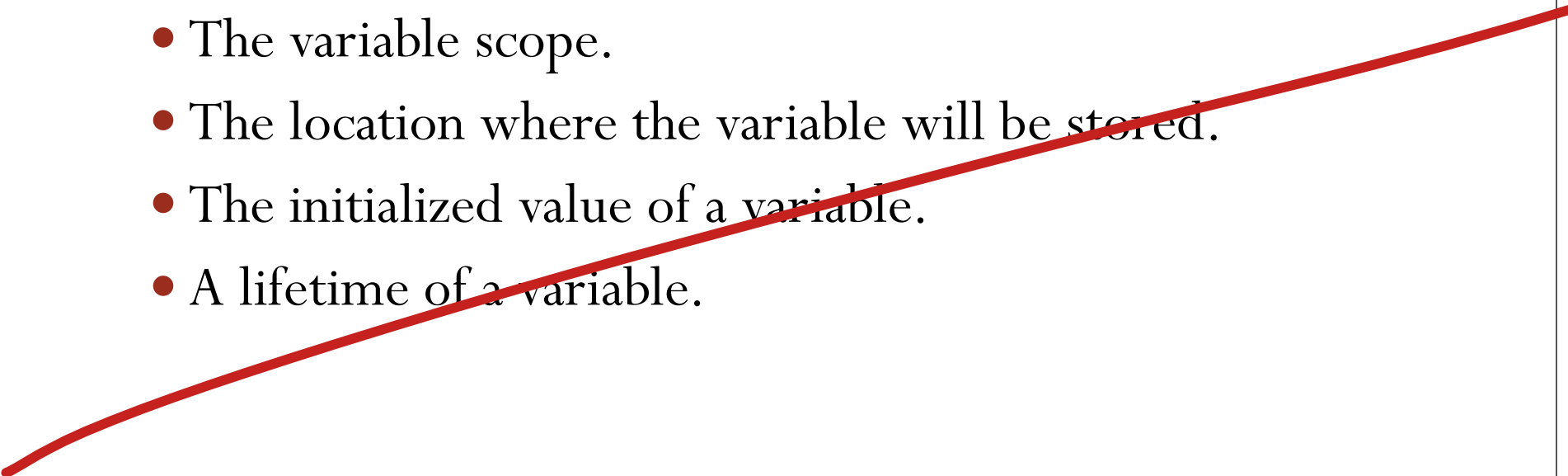
```
int prime(int n) {  
    int i;  
    for(i=2;i<=n/2;++i){  
        if(n%i==0) return 1;  
    }  
    return 0;  
}
```

input from user is passed to function prime() and integer value is returned from it. If the number is prime, 0 is returned and if number is not prime, 1 is returned.

Storage Classes

- Storage class specifies
 - Storage duration – how long an object exists in memory
 - Scope – where object can be referenced in program
 - Linkage – specifies the files in which an identifier is known
- Automatic storage
 - Object created and destroyed within its block
 - **auto**: default for local variables
 - `auto double x, y;`
 - **register**: tries to put variable into high-speed registers
 - Can only be used for automatic variables
 - `register int counter = 1;`

Storage Classes

- A storage class represents the visibility and a location of a variable.
 - It tells from what part of code we can access a variable.
 - A storage class is used to describe the following things:
 - The variable scope.
 - The location where the variable will be stored.
 - The initialized value of a variable.
 - A lifetime of a variable.
- 

Storage Classes

- There are total four types of standard storage classes.
- The table below represents the storage classes in 'C'.

Storage class	Purpose
auto	It is a default storage class.
extern	It is a global variable.
static	It is a local variable which is capable of returning a value even when control is transferred to the function call.
register	It is a variable which is stored inside a Register.

The **auto** Storage Class

- The **auto** storage class is the default storage class for all local variables.
- `int count;`
- `auto int count;`
- The example above defines two variables with in the same storage class. 'auto' can only be used within functions, i.e., local variables.

The **extern** Storage Class

- Variables that are declared outside of all functions are known as external or global variables.
- They are accessible from any function inside the program.

```
#include <stdio.h>
int n = 5;          // global variable
void display() { ++n;
                   printf("n = %d", n); }

int main() {
    ++n;
    display();
    return 0;
}
```

The **static** Storage Class

- Static local variable is a local variable that retains and stores its value between function calls and remains visible only to the function in which it is defined.
- Static global variables are global variables visible **only to the file in which it is declared.**
- Example: `static int count = 10;`
- Keep in mind that static variable has a default initial value zero and is initialized only once in its lifetime.

The **static** Storage Class

```
#include <stdio.h>

void display() {
    static int c = 1;
    c += 5;
    printf("%d ",c);
}.

int main() {
    display();
    display();
}
```

Output:
6 11

The **register** Storage Class

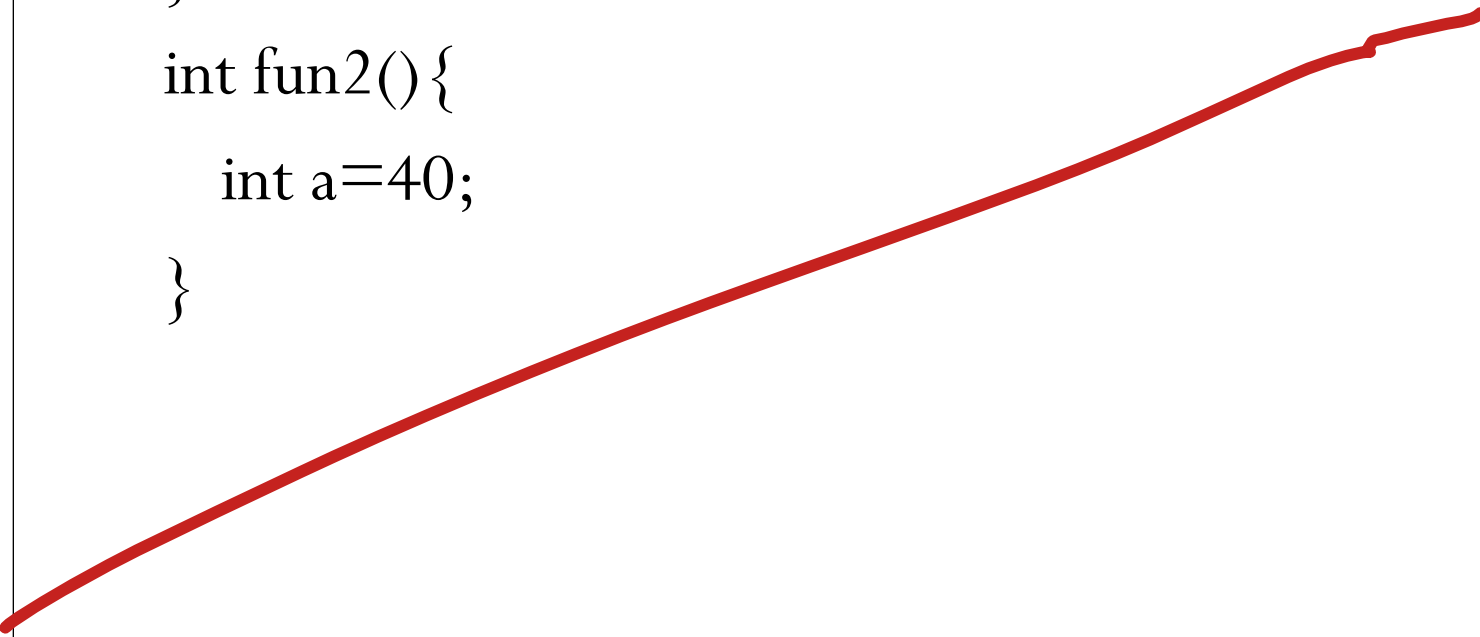
- The **register** keyword is used to declare register variables.
- Register variables were supposed to be faster than local variables.
 - However, modern compilers are very good at code optimization, and there is a rare chance that using register variables will make your program faster.
- Unless you are working on embedded systems where you know how to optimize code for the given application, there is no use of register variables.

Scope

- The scope of a variable is the section of the program within which the variable can be accessed
- Scoping is necessary if you can reuse the variable name.
- The scope of an automatic variable (local variable) is the function within which it is declared
- The scope of an external variable is from where it is declared to the end of the file
- Automatic variables with the same name in different functions are unrelated

Scope : same variable name in different functions

```
int fun1() {  
    int a=10;  
}  
int fun2() {  
    int a=40;  
}
```



Types of Function Calls

- **Call by value**
 - Copy of argument passed to function
 - Changes in function do not affect original
 - Use when function does not need to modify argument
 - Avoids accidental changes

Example

Write a program to calculate and print the area and the perimeter of a circle. Note that the radius is to be entered by the user. (Use *Call by value* approach)

```
#include<stdio.h>    /*The function calls are Call by Value*/
#define pi 3.14
float area(float);
float perimeter(float);
int main( )
{
    float r, a, p;
    printf("Enter the radius\n");
    scanf("%f",&r);
    a = area(r);
    p = perimeter (r);
    printf("The area = %.2f, \n The Perimeter = %.2f", a, p);
    return 0;
```

Example

```
float area (float x) {  
    return pi*x*x;  
}
```

```
float perimeter (float x){  
    return 2.0*pi*x;  
}
```

TYPES OF FUNCTION CALLS

• Call by Reference:

When a function is called by an argument/parameter which is a pointer (address of the argument) the copy of *the address of the argument* is passed to the function.

Therefore, a possible change on the data at the referenced address change the original value of the argument.

- Passes original argument
- Changes in function effect original
- Only used with trusted functions

Example

Write a program to calculate and print the area and the perimeter of a circle.

Note that the radius is to be entered by the user. (Use *Call by reference*)

```
#include<stdio.h> /*The function calls is Call by Reference*/  
#define pi 3.14  
void area_perimeter(float, float *, float *);  
  
void main( ){  
    float r, a, p;  
    printf("Enter the radius\n");  
    scanf("%f",&r);  
    area_perimeter(r,&a,&p);  
    printf("The area = %.2f, \n The Perimeter = %.2f", a, p);  
}
```

Example

```
void area_perimeter(float x, float *aptr, float *pptr){  
    *aptr = pi*x*x;  
    *pptr = 2.0*pi*x;  
}
```

Swap Variable example

```
void interchange(int x1,int y1) ;
```

```
main() {  
    int x=50, y=70;  
    interchange(x,y);  
    printf("x=%d y=%d",x,y);  
}  
  
void interchange(int x1,int y1) {  
    int z1;  
    z1=x1;  
    x1=y1;  
    y1=z1;  
    printf("x1=%d y1=%d",x1,y1);  
}
```

Output

X1=70 y1=50

X=50 y=70

Swap Variable example: call by reference

```
void interchange(int *x1,int *y1);
```

```
main() {
```

```
    int x=50, y=70;
```

```
    interchange(&x,&y);
```

```
    printf("x=%d y=%d",x,y);
```

```
}
```

```
void interchange(int *x1,int *y1) {
```

```
    int z1;
```

```
    z1=*x1;
```

```
    *x1=*y1;
```

```
    *y1=z1;
```

```
    printf("**x=%d *y=%d",x1,y1);
```

```
}
```

Output

***X=70 *y=50**

X=70 y=50

Questions?